

Universidade Aberta

Laboratório de Programação | 21178

E-Fólio A

GreenTrack: Gestão de Jardins Comunitários

Deteção e Correção de Erros e Organização Modular de Código

João André Frade Marques - 2505102

Abril de 2026

1. Introdução

Este relatório diz respeito ao E-Fólio A da unidade curricular Laboratório de Programação cujo objetivo é identificar e corrigir erros presentes em excertos de código C fornecidos pelo enunciado, e reorganizar o programa monolítico numa arquitetura modular.

O sistema em análise, GreenTrack, destina-se à gestão de jardins comunitários, permitindo registar plantas, controlar regas e gerir tarefas, com persistência de dados em ficheiros CSV.

A abordagem seguida foi a de analisar cada excerto, identificando erros e a sua categoria, propor correções justificadas, e depois reorganizar o código corrigido em módulos com interfaces bem definidas e encapsulamento adequado.

Tarefa 1 - Análise e Correção de Erros

Foram identificados 16 erros nos excertos fornecidos. A tabela seguinte detalha cada erro, as suas consequências e a correção aplicada.

Todas as correções implementadas no código fonte encontram-se devidamente comentadas com o número do erro referente a esta tabela.

Localização		Categoria	Consequências	Correção
1	carregar_dados()	Lógico	while(!feof()) executa iteração extra após EOF, lê lixo e incrementa total_plantas.	Substituir por while(fscanf(...) == 6)
2	carregar_dados()	Buffer overflow	Sem verificação de total_plantas >= MAX_PLANTAS antes de escrever no array, o que pode levar a buffer overflow	Adicionar if(total_plantas >= MAX_PLANTAS) break; dentro do ciclo.
3	listar_plantas()	Lógico	i <= total_plantas acede a plantas[total_plantas], fora do array o que leva a comportamento indefinido.	Corrigir para i < total_plantas.
4	registar_rega()	Buffer overflow	Sem verificação de total_regas >= MAX_REGAS antes de escrever, pode levar a buffer overflow	Adicionar if(total_regas >= MAX_REGAS) return 0; no início.
5	registar_rega()	Lógico	Não atualiza ultima_rega da planta após registar uma rega, por isso a função verificar_rega vais mostrar dados desatualizados	Após gravar a rega, atualizar o campo ultima_rega da planta.
6	criar_tarefa()	Buffer overflow	Sem verificação de total_tarefas >= MAX_TAREFAS, pode levar a buffer overflow	Adicionar if(total_tarefas >= MAX_TAREFAS) return; no início.
7	criar_tarefa()	Buffer overflow	strcpy sem limite. Se descricao tiver mais de 99 chars, escreve além do array.	Garantir terminação nula e adicionar limite (99)
8	listar_tarefas_pendentes()	Lógico	if(tarefas[i].concluida = 0) atribui 0, condição é sempre falsa	Corrigir para if(tarefas[i].concluida == 0).

Localização		Categoria	Consequências	Correção
9	concluir_tarefa()	Lógico	if(tarefas[i].id_tarefa = id). Se id != 0 entra sempre	Corrigir para if(tarefas[i].id_tarefa == id).
10	guardar_dados()	Gestão de ficheiros	Não guarda regas nem tarefas e esses dados perdem-se ao reiniciar o programa.	Acrescentar código equivalentes para regas.csv e tarefas.csv.
11	guardar_dados()	Integridade de dados	Campos de texto com vírgulas corrompem o formato CSV no fprintf.	Envolver campos de texto em aspas: "\"%s\"".
12	adicionar_planta()	Sintaxe	plantas[n].ultima_rega total_plantas++ é uma expressão inválida, falta o operador de atribuição =.	Corrigir para plantas[n].ultima_rega = 0; total_plantas++;
13	adicionar_planta()	Buffer overflow	Sem verificação de total_plantas >= MAX_PLANTAS antes de adicionar.	Adicionar if(total_plantas >= MAX_PLANTAS) return; no início.
14	main() — scanf	Segurança	scanf("%s", nome) sem limite de tamanho causa overflow se input > tamanho do buffer.	Usar scanf(" %49[^\n]", nome)
15	main() — scanf	Funcional	scanf("%s") para nomes com espaços	Usar fgets para ler input com espaços.
16	main()	Sintaxe	Falta a palavra "do" antes do switch. O do while está incompleto e não compila.	Corrigir para do { switch(opcao) { ... } } while(opcao != 8);

Erros agrupados

Ao efetuar a reorganização modular, podemos verificar que existem erros que vão repetir nos diferentes módulos. O erro 1, por exemplo, iria verificar-se sempre que se tentassem carregar dados, quer fosse no módulo plantas, tarefas ou regas.

Da mesma maneira podemos agrupar os erros 2, 4, 6 e 13. Esta violação de pré-condições, é transversal a todos os módulos onde é escrito ou lido em ficheiros .csv.

Os erros 8 e 9 também são semelhantes, pois consistem no mesmo erro de utilizar o símbolo de atribuição "=" e vez de comparação "==" nos respetivos ciclos if, mas trazem consequências diferentes, detalhadas na tabela.

Uma situação que também pode ser considerada um erro, é a não validação das datas introduzidas. Impor uma validação de forma rígida par dia mês e ano corrigia esta situação, mas para evitar complexidade adicional e fora do contexto do exercício, manteve-se o formato original.

Também foram convertidas algumas funções definidas como void para devolver um valor do tipo int. Assim, podemos ter indicação explícita se uma operação foi concluída com sucesso ou não. Mesmo não sendo utilizada no imediato, esta modificação prepara o código para extensões futuras, como melhor gestão de erros.

Tarefa 2 – Reorganização Modular

2a – Para esta tarefa, optei por uma divisão modular semelhante à utilizada nas atividades formativas, dividindo o projeto em duas pastas: **src**, que contém todos os ficheiros de implementação, e **include**, que contém todas as interfaces públicas.

Reorganizando os excertos utilizando esta divisão, ficamos com os seguintes módulos:

plantas, regas, tarefas, dados, e main. Cada módulo contém um ficheiro de implementação (.c) e um ficheiro de interface (.h).

2b –

Módulo Plantas:

Funções Publicas em plantas.h:

```
void listar_plantas();
int adicionar_planta(char* nome, char* especie, char* data_plantio, int intervalo);
int guardar_plantas(void);
int carregar_plantas(void);
int atualizar_ultima_rega(int id_planta, int data);
int planta_existe(int id_planta);
void verificar_rega(int data_atual);
```

Estas funções definem a interface publica do modulo e permitem a gestão de plantas sem acesso direto a variaveis.

Dentro de plantas.c temos como static a declaração do array plantas e a variável de contagem total_plantas. O acesso a estes dados apenas pode ser feito através de funções publicas, garantindo o encapsulamento, garantindo que nenhum outro modulo conhece ou manipula diretamente as plantas.

```
static Planta plantas[MAX_PLANTAS];
static int total_plantas = 0;

// Gerar ID sem input do utilizador
static int gerar_id_planta(void) {
    return total_plantas + 1;
}
```

Módulo Regas:

Funções Publicas em regas.h:

```
#ifndef REGAS_H
#define REGAS_H

int registrar_rega(int id_planta, int data, int quantidade);
int guardar_regas(void);
int carregar_regas(void);

#endif
```

Estas funções fornecem todas as operações necessárias para a gestão de regas e a sua persistencia sem expor as variáveis em regas.c

```
static Rega regas[MAX_REGAS];
static int total_regas = 0;
```

Neste módulo o encapsulamento é assegurado através destas variaveis static, como da ausencia de acesso direto às estruturas de plantas. A comunicação com o módulo plantas é feita exclusivamente através de funções públicas.

Módulo Tarefas:

Este módulo foi feito com uma abordagem semelhante a Regas.

Em tarefas.h podemos encontrar funções públicas que permitem criar, listar e concluir tarefas.

```
int criar_tarefa(char* descricao, int data_prevista);
void listar_tarefas_pendentes(void);
void concluir_tarefa(int id);
```

Mais uma vez, existe um isolamento completo de estruturas internas:

```
static int total_tarefas = 0;
static Tarefa tarefas[MAX_TAREFAS];

static int gerar_id_tarefa(void) {
    return total_tarefas + 1;
}
```

O encapsulamento fica assim garantido, pois o acesso a tarefas apenas pode ser feito pelas funções públicas.

Types.h:

Este ficheiro contém apenas a definição de estruturas e constantes, não incluindo qualquer variável global ou estado.

A exposição dos tipos não viola o encapsulamento, uma vez que apenas descreve a forma dos dados e não permite nenhum tipo de acesso ou modificação dos seus conteúdos.

Módulo Dados:

Este módulo foi criado apenas por uma questão de simplicidade no que toca a guardar dados, quer sejam tarefas, regas ou plantas. Não possui qualquer estrutura de dados própria e não gere quaisquer dados.

Como este módulo apenas invoca funções públicas, também não viola o encapsulamento.

Módulo Main:

Este módulo apenas contém a função main. Tal como Dados, limita-se apenas a chamar funções públicas de outros módulos, pelo que, também não viola o encapsulamento.

2c –

Para esta tarefa, decidi optar por esta reorganização modular por achar que havia um enquadramento claro com a tarefa. Existem funções claras e bem definidas, que lidam com três domínios: regas, plantas e tarefas. Ao dividir o código nestes módulos existe uma clara divisão de responsabilidades, e torna o código mais intuitivo e fácil de compreender.

Uma alternativa que foi considerada foi usar um ficheiro, por exemplo, `funcoes.c` para gerir todas as funções e depois aceder a essas funções através de main.

Nesta abordagem, esse ficheiro iria tornar-se demasiado grande e complexo, e ao mesmo tempo misturar responsabilidades distintas dentro do mesmo ficheiro, tornando-o demasiado confuso e complexo.

Com o método de reorganização modular utilizado, a gestão de dependências pode ser bem controlada:

- Main depende apenas dos headers públicos de cada módulo.
- Regas depende apenas da interface pública de Plantas
- Tarefas é independente dos outros módulos
- Dados usa apenas funções públicas de carregamento e gravação

Conclusão

A reorganização modular trouxe benefícios claros: separação de responsabilidades, encapsulamento efetivo dos dados, interfaces bem definidas e facilidade de manutenção futura. O maior desafio foi garantir que nenhum módulo acede diretamente aos arrays de outro, o que obrigou a desenhar funções de acesso adequadas.

Este trabalho também permitiu observar a importancia de boas práticas, e como facilmente um erro numa função se pode propagar a diferentes módulos.

Como aluno que apenas ingressou no segundo semestre foi tambem um desafio conseguir ficar a par de toda a matéria, tendo que rever vários conceitos da linguagem de programação C. Através das atividades formativas posso dizer que me senti preparado para esta tarefa.

Julgo que existem melhorias a fazer, quer na componente prática como pessoal.

Esta avaliação fez me perceber que existem sempre pequenos detalhes a corrigir e melhorias a ser feitas para atingir um bom resultado final. Ao mesmo tempo sinto que não geri o meu tempo da melhor maneira, às vezes perdendo o foco em tarefas que não são cruciais.

Código Completo

Código completo com as respectivas correções e arquitetura modular implementada.

Main.c

```
#include <stdio.h>
#include <stdlib.h>
#include "../include/types.h" // Acesso a definições de tipos
#include "../include/dados.h" // Acesso a funções de carregar e guardar dados
#include "../include/plantas.h" // Acesso a funções relacionadas a plantas
#include "../include/regas.h" // Acesso a funções relacionadas a regas
#include "../include/tarefas.h" // Acesso a funções relacionadas a tarefas

/* MAIN */
// Erro 14, scanf sem limites de tamanho (em todos)

int main(void) {

    carregar_dados(); // Carregar os dados existentes ao iniciar o programa

    int opcao;
    printf("Bem-vindo ao sistema de gestão de plantas!\n");

    // Correção do Erro 16: Sintaxe, do while (Do em falta)
    do{

        /* MENU */
        printf("1. Listar plantas\n");
        printf("2. Adicionar planta\n");
        printf("3. Listar regas\n");
        printf("4. Adicionar rega\n");
        printf("5. Listar tarefas\n");
        printf("6. Adicionar tarefa\n");
        printf("7. Concluir tarefa\n");
        printf("8. Sair\n");
        printf("Escolha uma opcao: ");
        scanf("%d", &opcao);

        switch (opcao) { // Correção = adicionar do antes do switch e fechar o
        bloco do while no final do main

            // Listar Plantas
            case 1:{
                printf("\n=== LISTA DE PLANTAS ===\n");
                listar_plantas();
                printf("Prima Enter para voltar ao Menu principal... ");
                getchar(); // Limpar o buffer do teclado
                getchar(); // Esperar pela entrada do utilizador
                break; // Voltar ao menu
```

```

}

// Adicionar Planta
case 2: {
    printf("\n=== ADICIONAR PLANTAS ===\n");
    getchar();

    char nome[50], especie[50], data_plantio[11];
    int intervalo_rega; /* em dias */

    printf("Introduza o Nome: ");
    //Correção do Erro 14: Substituir scanf por fgets para ler strings com
    espaços e evitar buffer overflow.
    // Esta correção repete-te para todo o main onde existia scanf para ler
    strings.
    fgets(nome, sizeof(nome), stdin);
    nome[strcspn(nome, "\n")] = '\0';

    printf("Introduza a Especie: ");
    fgets(especie, sizeof(especie), stdin);
    especie[strcspn(especie, "\n")] = '\0';

    printf("Introduza a Data de Plantio (DD/MM/AAAA): ");
    fgets(data_plantio, sizeof(data_plantio), stdin);
    data_plantio[strcspn(data_plantio, "\n")] = '\0';

    printf("Introduza o Intervalo de Rega (em dias): ");
    scanf("%d", &intervalo_rega);

    adicionar_planta(nome, especie, data_plantio, intervalo_rega);
    printf("Planta adicionada com sucesso!\n");
    guardar_dados();
    break;
}

case 3:{
    // Verificar Regas
    printf("\n=== VERIFICAR NECESSIDADE DE REGA ===\n");
    int data_atual;
    printf("Data atual (dias desde 01/01/2026): ");
    scanf("%d", &data_atual);
    printf("Prima Enter para voltar ao Menu principal... ");
    getchar();
    getchar();
    break;
}

case 4: {
    // Adicionar Rega
    printf("\n=== ADICIONAR REGA ===\n");
    int id_planta, data, quantidade;

```

```

        printf("ID da planta: ");
        scanf("%d", &id_planta);
        printf("Data da rega (dias desde 01/01/2026): ");
        scanf("%d", &data);
        printf("Quantidade de agua (em ml): ");
        scanf("%d", &quantidade);
        registrar_rega(id_planta, data, quantidade);
        guardar_dados();
        break;
    }

    case 5: {
        // Listar Tarefas
        listar_tarefas_pendentes();
        printf("Prima Enter para voltar ao Menu principal... ");
        getchar();
        getchar();
        break;
    }

    case 6: {
        // Adicionar Tarefa
        printf("\n=== ADICIONAR TAREFA ===\n");
        getchar();

        char descricao[100];
        int data_prevista;

        printf("Descricao da tarefa: ");
        fgets(descricao, sizeof(descricao), stdin);
        descricao[strlen(descricao)] = '\0';

        printf("Data prevista (dias desde 01/01/2026): ");
        scanf("%d", &data_prevista);

        criar_tarefa(descricao, data_prevista);
        guardar_dados();

        break;
    }

    case 7: {
        // Concluir Tarefa
        printf("\n=== CONCLUIR TAREFA ===\n");
        int id;
        printf("ID da tarefa a concluir: ");
        scanf("%d", &id);

        concluir_tarefa(id);
        guardar_dados();
    }
}

```

```

        break;
    }

    case 8: {
        printf("A sair do sistema. Até logo!\n");
        return 0; // sai do programa
    }

    default: {
        printf("Opcao invalida. Tente novamente.\n");
    }
} while (opcao != 8);

return 0;
}

```

Plantas.c

```

#include "../include/plantas.h"
#include "../include/types.h"
#include <stdio.h>
#include <string.h>

static Planta plantas[MAX_PLANTAS];
static int total_plantas = 0;

// Gerar ID sem input do utilizador
static int gerar_id_planta(void) {
    return total_plantas + 1;
}

/***** PLANTAS *****/

/* LISTAR AS PLANTAS */
void listar_plantas(void) {

    if (total_plantas == 0) {
        printf("Nao existem plantas registadas.\n");
        return;
    }

    // Corrige Erro 3: O ciclo for deve ser i < total_plantas
    for (int i = 0; i < total_plantas; i++) {
        printf("ID: %d\n", plantas[i].id);
        printf("  Nome: %s\n", plantas[i].nome);
        printf("  Especie: %s\n", plantas[i].especie);
        printf("  Data de plantio: %s\n", plantas[i].data_plantio);
        printf("  Intervalo de rega: %d dias\n", plantas[i].intervalo_rega);
        printf("  Ultima rega: %d\n", plantas[i].ultima_rega);
        printf("-----\n");
    }
}

```

```

/* ADICIONAR PLANTA */

int adicionar_planta(char* nome, char* especie, char* data_plantio, int intervalo)
{
    // Corrige Erro 13: Verificar se o limite de plantas não foi ultrapassado, o
    // que pode causar um buffer overflow
    if (total_plantas >= MAX_PLANTAS) {
        return 0;
    }
    plantas[total_plantas].id = gerar_id_planta();

    strncpy(plantas[total_plantas].nome, nome, sizeof(plantas[total_plantas].nome)
- 1);
    plantas[total_plantas].nome[sizeof(plantas[total_plantas].nome) - 1] = '\0'; //
    Garantir terminação nula

    strncpy(plantas[total_plantas].especie, especie,
    sizeof(plantas[total_plantas].especie) - 1);
    plantas[total_plantas].especie[sizeof(plantas[total_plantas].especie) - 1] =
    '\0'; // Garantir terminação nula

    strncpy(plantas[total_plantas].data_plantio, data_plantio,
    sizeof(plantas[total_plantas].data_plantio) - 1);
    plantas[total_plantas].data_plantio[sizeof(plantas[total_plantas].data_plantio)
- 1] = '\0'; // Garantir terminação nula

    plantas[total_plantas].intervalo_rega = intervalo;
    plantas[total_plantas].ultima_rega = 0; // Corrige Erro 12: Inicializar ultima
    rega como 0 (nunca regada)

    total_plantas++;
    return 1;
}

/* GUARDAR PLANTAS */

// Int em vez de void para poder retornar um código de erro.
int guardar_plantas(void) {
    FILE *f_plantas = fopen("plantas.csv", "w");
    if (f_plantas == NULL) {
        return 0;
    }

    for (int i = 0; i < total_plantas; i++) {
        fprintf(f_plantas, "%d,%s,%s,%s,%d,%d\n",
            plantas[i].id,
            plantas[i].nome,
            plantas[i].especie,

```

```

        plantas[i].data_plantio,
        plantas[i].intervalo_rega,
        plantas[i].ultima_rega);
    }
    fclose(f_plantas);
    return 1;
}

/* CARREGAR PLANTAS */

int carregar_plantas(void) {
    FILE *f_plantas = fopen("plantas.csv", "r");
    if (f_plantas == NULL) {
        return 0;
    }

    total_plantas = 0;
    // Correção do erro 1: ciclo while corrigido do enunciado. Corrige while
    (!feof(f_plantas))
        while (total_plantas < MAX_PLANTAS && fscanf(f_plantas,
"%d,%[^,],%[^,],%[^,],%d,%d\n", // Correção do erro 2: Verificar se não existe
buffer overflow
        &plantas[total_plantas].id,
        plantas[total_plantas].nome,
        plantas[total_plantas].especie,
        plantas[total_plantas].data_plantio,
        &plantas[total_plantas].intervalo_rega,
        &plantas[total_plantas].ultima_rega) == 6) {
            total_plantas++;
        }

    fclose(f_plantas);
    return 1;
}

/* VERIFICAR NECESSIDADE DE REGA */
void verificar_rega(int data_atual) {

    printf("=== PLANTAS QUE PRECISAM DE REGA ===\n");
    for (int i = 0; i < total_plantas; i++) {
        int dias = data_atual - plantas[i].ultima_rega;
        if (dias >= plantas[i].intervalo_rega) {
            // Possível erro no printf
            printf("Planta %s (ID: %d) precisa de rega! (ultima: %d dias atras)\n",
            plantas[i].nome, plantas[i].id, dias);
        }
    }
}

/* ATUALIZAR ULTIMA REGA */

```

```
// Permite a atualização da data da ultima rega sem expor a variavel diretamente.
int atualizar_ultima_rega(int id_planta, int data){
    for (int i = 0; i < total_plantas; i++) {
        if (plantas[i].id == id_planta) {
            plantas[i].ultima_rega = data;
            return 1;
        }
    }
    return 0;
}

// VERIFICAR SE PLANTA EXISTE
int planta_existe(int id_planta) {
    for (int i = 0; i < total_plantas; i++) {
        if (plantas[i].id == id_planta) {
            return 1;
        }
    }
    return 0;
}
```

Plantas.h

```
#ifndef PLANTAS_H
#define PLANTAS_H
#include "../include/types.h"

void listar_plantas(void);
int adicionar_planta(char* nome, char* especie, char* data_plantio, int intervalo);
int guardar_plantas(void);
int carregar_plantas(void);
int atualizar_ultima_rega(int id_planta, int data);
int planta_existe(int id_planta);
void verificar_rega(int data_atual);

#endif
```

Regas.c

```
#include <stdio.h>
#include "../include/regas.h"
#include "../include/plantas.h" // Acesso a funções relacionadas a plantas
#include "../include/types.h" // Acesso a definições de tipos e constantes

static Rega regas[MAX_REGAS];
static int total_regas = 0;

/* REGISTRAR REGAS */
int registrar_rega(int id_planta, int data, int quantidade){
```

```

// Verificar se a planta existe
if(!planta_existe(id_planta)){
    printf("Erro: Planta com ID %d nao existe.\n", id_planta);
    return 0;
}

// Correção do erro 4: Verificar o limite de regas
if(total_regas >= MAX_REGAS){
    printf("Erro: Limite de regas atingido. Nao e possivel registrar mais
regas.\n");
    return 0;
};

regas[total_regas].id_rega = total_regas + 1;
regas[total_regas].id_planta = id_planta;
regas[total_regas].data_rega = data;
regas[total_regas].quantidade_agua = quantidade;
total_regas++;

// Correção do Erro 5: Atualiza a ultima rega da planta após registrar a rega,
garantindo que a função verificar_regas() funcione corretamente.
atualizar_ultima_rega(id_planta, data);
printf("Rega registrada com sucesso!\n");
return 1;
}

/* GUARDAR REGAS */
int guardar_regas(void) {
    FILE *f_regas = fopen("regas.csv", "w");
    if (f_regas == NULL) {
        return 0;
    }

    for (int i = 0; i < total_regas; i++) {
        fprintf(f_regas, "%d,%d,%d,%d\n",
            regas[i].id_rega,
            regas[i].id_planta,
            regas[i].data_rega,
            regas[i].quantidade_agua);
    }
    fclose(f_regas);
    return 1;
}

/* CARREGAR REGAS */
int carregar_regas(void) {
    FILE *f_regas = fopen("regas.csv", "r");
    if (f_regas == NULL) {
        return 0;
    }
}

```



```

    total_regas = 0;
    // ciclo while corrigido do enunciado. Corrige while (!feof(f_regas))
    while (fscanf(f_regas, "%d,%d,%d,%d\n",
                &regas[total_regas].id_rega,
                &regas[total_regas].id_planta,
                &regas[total_regas].data_rega,
                &regas[total_regas].quantidade_agua) == 4 && total_regas <
MAX_REGAS) {

        total_regas++;
    }

    fclose(f_regas);
    return 1;
}

```

Regas.h

```

#ifndef REGAS_H
#define REGAS_H

int registrar_rega(int id_planta, int data, int quantidade);
int guardar_regas(void);
int carregar_regas(void);

#endif

```

Tarefas.c

```

#include "../include/tarefas.h"
#include "../include/types.h"
#include <stdio.h>

static int total_tarefas = 0;
static Tarefa tarefas[MAX_TAREFAS];

static int gerar_id_tarefa(void) {
    return total_tarefas + 1;
}

int criar_tarefa(char* descricao, int data_prevista) {

    // Correção do Erro 6: Verificar se o limite de tarefas não foi ultrapassado
    if(total_tarefas >= MAX_TAREFAS) {
        printf("Erro: Limite de tarefas atingido.\n");
        return 0;
    }
}

```

```

    tarefas[total_tarefas].id_tarefa = gerar_id_tarefa();
    // Correção do Erro 7: strcpy sem limite
    strncpy(tarefas[total_tarefas].descricao, descricao,
sizeof(tarefas[total_tarefas].descricao) - 1);
    tarefas[total_tarefas].descricao[sizeof(tarefas[total_tarefas].descricao) - 1]
= '\0'; // Garantir terminação nula
    tarefas[total_tarefas].data_prevista = data_prevista;
    tarefas[total_tarefas].concluida = 0;
    total_tarefas++;

    printf("Tarefa criada com sucesso!\n");
    return 1;
}

/* LISTAR TAREFAS */

void listar_tarefas_pendentes(void) {

    if(total_tarefas == 0) {
        printf("Nao existem tarefas registradas.\n");
        return;
    }
    for (int i = 0; i < total_tarefas; i++) {
        // Correção do Erro 8: O operador de comparação deve ser ==, mas foi usado
        // =, o que causa uma atribuição em vez de uma comparação, resultando em um
        // comportamento incorreto (sempre entrando no if e marcando a tarefa como concluida)
        if (tarefas[i].concluida == 0) {
            printf("Tarefa %d: %s (prevista: %d)\n",
tarefas[i].id_tarefa, tarefas[i].descricao,
tarefas[i].data_prevista);
        }
    }
}

/* CONCLUIR TAREFA */

void concluir_tarefa(int id) {
    for (int i = 0; i < total_tarefas; i++) {
        // Correção do Erro 9: O operador de comparação deve ser ==, mas foi usado
        // =, o que causa uma atribuição em vez de uma comparação, resultando em um
        // comportamento incorreto (sempre marcando a tarefa como concluida independentemente
        // do id)
        if (tarefas[i].id_tarefa == id) {
            tarefas[i].concluida = 1;
            printf("Tarefa concluida com sucesso!\n");
            return;
        }
    }
    printf("Tarefa nao encontrada.\n");
}

```

```

/* GUARDAR TAREFAS */

int guardar_tarefas(void) {
    // Correção do Erro 10: Guardar tarefas em um arquivo CSV, garantindo que os
    dados sejam persistidos corretamente
    FILE *f_tarefas = fopen("tarefas.csv", "w");
    if (f_tarefas == NULL) {
        return 0;
    }
    // Corrige Erro 11
    for (int i = 0; i < total_tarefas; i++) {
        fprintf(f_tarefas, "%d,%s,%d,%d\n",
            tarefas[i].id_tarefa,
            tarefas[i].descricao,
            tarefas[i].data_prevista,
            tarefas[i].concluida);
    }
    fclose(f_tarefas);
    return 1;
}

/* CARREGAR TAREFAS */

int carregar_tarefas(void) {

    FILE *f_tarefas = fopen("tarefas.csv", "r");
    if (f_tarefas == NULL) {
        return 0;
    }

    total_tarefas = 0;
    // Erro 1: ciclo while corrigido do enunciado. Corrige while (!feof(f_tarefas))
    while (fscanf(f_tarefas, "%d,%99[^\n],%d,%d\n",
        &tarefas[total_tarefas].id_tarefa,
        tarefas[total_tarefas].descricao,
        &tarefas[total_tarefas].data_prevista,
        &tarefas[total_tarefas].concluida) == 4) {

        total_tarefas++;

        if (total_tarefas >= MAX_TAREFAS) {
            printf("Aviso: Limite de tarefas atingido ao carregar dados.\n");
            break;
        }
        if (total_tarefas == 0) {
            printf("Nao existem tarefas registradas.\n");
            break;
        }
    }
}

```

```
    fclose(f_tarefas);  
    return 1;  
}
```

Tarefas.h

```
#ifndef TAREFAS_H  
#define TAREFAS_H  
#include "../include/types.h"  
  
int criar_tarefa(char* descricao, int data_prevista);  
void listar_tarefas_pendentes(void);  
void concluir_tarefa(int id);  
  
#endif
```

Dados.c

```
#include "../include/dados.h"  
#include "../include/plantas.h"  
#include "../include/regas.h"  
#include "../include/tarefas.h"  
  
void carregar_dados(void) {  
    carregar_plantas();  
    carregar_regas();  
    carregar_tarefas();  
}  
  
void guardar_dados(void) {  
    guardar_plantas();  
    guardar_regas();  
    guardar_tarefas();  
}
```

Dados.h

```
#ifndef DADOS_H  
#define DADOS_H  
  
void carregar_dados(void);  
void guardar_dados(void);  
int carregar_regas(void);  
int guardar_regas(void);  
int carregar_tarefas(void);  
int guardar_tarefas(void);  
  
#endif
```

Types.h

```
/* TYPES AND VARIABLES */

#ifndef TYPES_H
#define TYPES_H
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* — Tipos de dados — */
#define MAX_PLANTAS 100
#define MAX_REGAS 500
#define MAX_TAREFAS 200

typedef struct {
int id;
char nome[50];
char especie[50];
char data_plantio[11]; /* DD/MM/AAAA */
int intervalo_rega; /* em dias */
int ultima_rega; /* timestamp: dias desde 01/01/2026 */
} Planta;

typedef struct {
int id_rega;
int id_planta;
int data_rega; /* timestamp: dias desde 01/01/2026 */
int quantidade_agua; /* em ml */
} Rega;

typedef struct {
int id_tarefa;
char descricao[100];
int data_prevista; /* timestamp: dias desde 01/01/2026 */
int concluida; /* 0 = pendente | 1 = concluida */
} Tarefa;

#endif
```